

Skill-8

Task -1:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>

void show_usage()
{
    printf("Usage:\n");
    printf(" ./combined_redirection output.txt\n");
}

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        show_usage();
        return 1;
    }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
if (fd < 0)
{
    perror("open");
    return 1;
}
pid_t pid = fork();
if (pid < 0)
{
    perror("fork");
    close(fd);
    return 1;
}
if (pid == 0)
{
    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 stdout");
        exit(1);
    }
    if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0)
    {
        perror("dup2 stderr");
        exit(1);
    }
    close(fd);
    printf("Child: stdout message\n");
```

```

    fflush(stdout);

    fprintf(stderr, "Child: stderr message\n");

    fflush(stderr);

    printf("Child: second stdout message\n");

    fflush(stdout);

    fprintf(stderr, "Child: second stderr message\n");

    fflush(stderr);

    exit(0);
}

close(fd);

waitpid(pid, NULL, 0);

printf("Parent: child execution completed\n");

return 0;
}

```

Output:

```

root@Rahul:~# mkdir -p ~/OSSP/task1
root@Rahul:~# cd ~/OSSP/task1
root@Rahul:~/OSSP/task1# nano combined_redirection.c
root@Rahul:~/OSSP/task1# gcc combined_redirection.c -o combined_redirection
root@Rahul:~/OSSP/task1# ls
combined_redirection  combined_redirection.c
root@Rahul:~/OSSP/task1# ./combined_redirection output.txt
Parent: child execution completed
root@Rahul:~/OSSP/task1# cat output.txt
Child: stdout message
Child: stderr message
Child: second stdout message
Child: second stderr message
root@Rahul:~/OSSP/task1# rm output.txt
root@Rahul:~/OSSP/task1# ./combined_redirection output.txt
Parent: child execution completed
root@Rahul:~/OSSP/task1# echo "OLD DATA" > output.txt
root@Rahul:~/OSSP/task1# ./combined_redirection output.txt
Parent: child execution completed
root@Rahul:~/OSSP/task1# ./combined_redirection /invalid/path/output.txt
open: No such file or directory
root@Rahul:~/OSSP/task1# |

```

Task-2:

To Parse Complex Commands, Combine Pipes and Redirections,
Resolve Execution Order, Validate Syntax, Generate Execution Plans,
Test Scenarios

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#define MAX_CMD 256
#define MAX_PARTS 20
void trim(char *str)
{
    char *start = str;
    while (*start == ' ' || *start == '\t')
        start++;
    memmove(str, start, strlen(start) + 1);
    int len = strlen(str);
    while (len > 0 && (str[len - 1] == ' ' || str[len - 1] == '\t'))
    {
        str[len - 1] = '\0';
        len--;
    }
}
int validate_command(char *cmd)
{
    if (strlen(cmd) == 0)
        return 0;
    if (strstr(cmd, "|") != NULL)
        return 0;
    if (strstr(cmd, ">") != NULL)
```

```

        return 0;

    if (cmd[0] == '|')
        return 0;

    int len = strlen(cmd);
    if (cmd[len - 1] == '|')
        return 0;

    return 1;
}

void print_plan(char *command)
{
    printf("\nExecution Plan\n");
    printf("-----\n");

    char temp[MAX_CMD];
    strcpy(temp, command);

    char *part = strtok(temp, "|");
    int number = 1;
    while (part != NULL)
    {
        trim(part);

        printf("Stage %d: %s\n", number, part);
        part = strtok(NULL, "|");
        number++;
    }
    printf("-----\n");
}

void execute_single(char *cmd)
{
    char *args[MAX_PARTS];
    char *token;

    int count = 0;

    char input_file[100] = "";
    char output_file[100] = "";
    char error_file[100] = "";

```

```
int append = 0;

token = strtok(cmd, " \\t");

while (token != NULL && count < MAX_PARTS - 1)
{
    if (strcmp(token, "<") == 0)
    {
        token = strtok(NULL, " \\t");

        if (token == NULL)
        {
            fprintf(stderr, "Missing input file\\n");

            return;
        }

        strcpy(input_file, token);
    }
    else if (strcmp(token, ">") == 0)
    {
        token = strtok(NULL, " \\t");

        if (token == NULL)
        {
            fprintf(stderr, "Missing output file\\n");

            return;
        }

        strcpy(output_file, token);

        append = 0;
    }
    else if (strcmp(token, ">>") == 0)
    {
        token = strtok(NULL, " \\t");

        if (token == NULL)
        {
            fprintf(stderr, "Missing output file\\n");

            return;
        }
    }
}
```

```

        strcpy(output_file, token);

        append = 1;
    }
    else if (strcmp(token, ">") == 0)
    {
        token = strtok(NULL, " \t");
        if (token == NULL)
        {
            fprintf(stderr, "Missing error file\n");
            return;
        }
        strcpy(error_file, token);
    }
    else
    {
        args[count++] = token;
    }
    token = strtok(NULL, " \t");
}
args[count] = NULL;
if (count == 0)
    return;
if (input_file[0] != '\0')
{
    int fd = open(input_file, O_RDONLY);
    if (fd < 0)
    {
        perror("input");
        exit(1);
    }
    dup2(fd, STDIN_FILENO);
    close(fd);
}

```

```

if (output_file[0] != '\0')
{
    int flags = O_WRONLY | O_CREAT;
    if (append)
        flags |= O_APPEND;
    else
        flags |= O_TRUNC;
    int fd = open(output_file, flags, 0644);
    if (fd < 0)
    {
        perror("output");
        exit(1);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
}

if (error_file[0] != '\0')
{
    int fd = open(error_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0)
    {
        perror("error");
        exit(1);
    }
    dup2(fd, STDERR_FILENO);
    close(fd);
}

execvp(args[0], args);
perror("execvp");
exit(1);
}

void execute_pipeline(char *command)
{

```

```
char *commands[MAX_PARTS];

int count = 0;

char *token = strtok(command, "|");

while (token != NULL && count < MAX_PARTS)
{
    trim(token);

    commands[count++] = token;

    token = strtok(NULL, "|");
}

int previous_fd = -1;

pid_t pids[MAX_PARTS];

for (int i = 0; i < count; i++)
{
    int pipefd[2];

    if (i < count - 1)
    {
        if (pipe(pipefd) < 0)
        {
            perror("pipe");

            return;
        }
    }

    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");

        return;
    }

    if (pids[i] == 0)
    {
        if (previous_fd != -1)
        {
            dup2(previous_fd, STDIN_FILENO);
```

```

        close(previous_fd);
    }
    if (i < count - 1)
    {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
    }
    execute_single(commands[i]);
    exit(1);
}
if (previous_fd != -1)
    close(previous_fd);
if (i < count - 1)
{
    close(pipefd[1]);
    previous_fd = pipefd[0];
}
}
for (int i = 0; i < count; i++)
    waitpid(pids[i], NULL, 0);
}

int main()
{
    char command[MAX_CMD];
    printf("Complex Command Executor\n");
    printf("Type 'exit' to quit\n");
    while (1)
    {
        printf("cmd> ");
        fflush(stdout);
        if (fgets(command, sizeof(command), stdin) == NULL)
            break;
    }
}

```

```
command[strcspn(command, "\n")] = '\0';

trim(command);

if (strcmp(command, "exit") == 0)

    break;

if (!validate_command(command))

{

    printf("Syntax Error\n");

    continue;

}

print_plan(command);

char execution[MAX_CMD];

strcpy(execution, command);

if (strchr(execution, '|') != NULL)

    execute_pipeline(execution);

else

{

    pid_t pid = fork();

    if (pid == 0)

    {

        execute_single(execution);

    }

    return 0;

}
```

Output:

```
root@Rahul:~/OSSP/task1# cd ~/OSSP
root@Rahul:~/OSSP# mkdir -p task2
root@Rahul:~/OSSP# cd task2
root@Rahul:~/OSSP/task2# nano complex_commands.c
root@Rahul:~/OSSP/task2# gcc complex_commands.c -o complex_commands
root@Rahul:~/OSSP/task2# ./complex_commands
Complex Command Executor
Type 'exit' to quit
cmd> ./complex_commands

Execution Plan
-----
Stage 1: ./complex_commands
-----

Complex Command Executor
Type 'exit' to quit
cmd> echo Hello > output.txt

Execution Plan
-----
Stage 1: echo Hello > output.txt
-----

cmd> cat output.txt

Execution Plan
-----
Stage 1: cat output.txt
-----

Hello
cmd> cat output.txt

Execution Plan
-----
Stage 1: cat output.txt
-----

Hello
cmd> echo "Ubuntu Linux Shell" > input.txt

Execution Plan
-----
Stage 1: echo "Ubuntu Linux Shell" > input.txt
-----

cmd> exit
cmd> |
```